

CardioGuard AI — Project Documentation

End-to-End Machine Learning System for Cardiovascular Disease Risk Stratification

Subject: Machine Learning Project Evaluation	Architecture: Stacking Classifier Meta-Ensemble
Model ROC-AUC: 0.8048 (73.20% Test Accuracy)	Tech Stack: Python, FastAPI, Scikit-learn, XGBoost, React, Vite

1. Executive Summary & Problem Formulation

Cardiovascular diseases (CVDs) are the leading cause of mortality globally. Early detection of asymptomatic high-risk patients using routine clinical biomarkers allows preventive lifestyle and therapeutic interventions. This project presents **CardioGuard AI**, a production-grade machine learning application designed to assess 10-year cardiovascular disease risk from physiological, hemodynamic, and lifestyle indicators.

Core Objectives:

- Establish a leakage-free preprocessing and feature engineering pipeline on 70,000 clinical records.
- Implement and benchmark baseline models (Logistic Regression, Decision Tree) and modern gradient boosting algorithms (XGBoost, LightGBM, HistGradientBoosting).
- Engineer a heterogeneous **Stacking Ensemble Classifier** with a cross-validated meta-model achieving superior ROC-AUC (0.8048).
- Deploy an asynchronous REST API using **FastAPI** paired with an interactive **React + Vite** responsive dashboard.

2. Dataset & Preprocessing Pipeline

The model is trained on the Cardiovascular Disease Dataset comprising 70,000 patient records across 12 primary features.

Feature Name	Type	Clinical Description	Data Range / Units
age_years	Continuous	Patient age in full solar years (age in days / 365.25)	18 – 100 years
gender	Categorical	Biological sex	1: Female, 2: Male
height / weight	Continuous	Stature and mass	100-220 cm / 30-200 kg
ap_hi / ap_lo	Continuous	Systolic / Diastolic arterial blood pressure	50-250 / 30-150 mmHg
cholesterol	Ordinal	Total serum cholesterol level tier	1: Normal, 2: Above, 3: High
gluc	Ordinal	Fasting serum glucose level tier	1: Normal, 2: Above, 3: High
smoke / alco	Binary	Tobacco smoking / Alcohol consumption habits	0: No, 1: Yes
active	Binary	Regular physical activity commitment	0: Inactive, 1: Active
cardio (Target)	Binary	Presence or absence of cardiovascular disease	0: Healthy, 1: Diseased

Data Cleaning & Outlier Filtration:

1. **Physiological Consistency:** Filtered records where diastolic pressure exceeded systolic pressure ($ap_hi \leq ap_lo$).
2. **Extreme Outlier Removal:** Removed impossible blood pressures (<50 mmHg or >250 mmHg for systolic; <30 mmHg or >150 mmHg for diastolic).
3. **Anthropometric Bounds:** Constrained height to $[100, 220]$ cm and weight to $[30, 200]$ kg.
4. **Resulting Dataset:** 68,640 clean records (50.0% class balance maintained).

Leakage-Free Feature Engineering Pipeline:

To maximize predictive signal without causing data leakage, the following clinical features are computed dynamically inside an `sklearn.base.TransformerMixin` (`FeatureEngineer`):

- **Body Mass Index (BMI):** $\frac{\text{weight (kg)}}{(\text{height (m)})^2}$
- **Pulse Pressure (PP):** $\text{ap_hi} - \text{ap_lo}$ (measures arterial vascular stiffness)
- **Mean Arterial Pressure (MAP):** $\text{ap_lo} + \frac{\text{PP}}{3}$ (measures continuous organ perfusion)
- **Blood Pressure Ratio:** $\frac{\text{ap_hi}}{\text{ap_lo}}$
- **Interaction Features:** $\text{Age} \times \text{ap_hi}$, $\text{Cholesterol} \times \text{Glucose}$, $\text{BMI} \times \text{Age}$
- **Log Transformations:** $\ln(1 + \text{weight})$, $\ln(1 + \text{BMI})$ to normalize right-skewed distributions.
- **Infinite-Bounded Binning:** Robust categorization for age groups, BMI tiers, and systolic ranges using $[-\infty, \dots, \infty]$ to prevent NaN evaluation.

3. Machine Learning Algorithms & Benchmark Evaluation

Models were evaluated using **Repeated Stratified K-Fold Cross-Validation** (3 splits, 2 repeats, 6 folds total) to guarantee statistical robustness. A separate untouched 20% test partition (13,728 patients) was used for final validation.

Model Architecture	Classification Paradigm	CV Mean Acc	Test Acc	Test ROC-AUC	F1 Score	Role
Stacking Classifier Ensemble	Heterogeneous Meta-Learner	73.53% ±0.26%	73.20%	0.8048	0.7165	Production Model
HistGradientBoosting	Binned Tree Boosting	73.56% ±0.23%	73.15%	0.7999	0.7151	Base Estimator
XGBoost Classifier	Regularized Boosting (XGB)	73.50% ±0.27%	73.08%	0.7991	0.7145	Base Estimator
LightGBM Classifier	Leaf-wise Boosting (LGBM)	73.50% ±0.31%	73.09%	0.7991	0.7147	Base Estimator
Decision Tree (depth=5)	Single Decision Tree	—	73.11%	0.7919	0.7134	Baseline Benchmark
Logistic Regression (Sklearn)	L2 Linear Logistic Model	—	72.67%	0.7935	0.7070	Meta-Model
Logistic Regression (Scratch)	Custom Gradient Descent	—	72.70%	0.7920	0.7065	Algorithm Verification

Stacking Ensemble Architecture Explained:

Level-0 Base Learners: Three diverse gradient boosting algorithms (HistGradientBoosting, XGBoost, LightGBM) learn non-linear feature interactions and high-dimensional boundaries independently.

Level-1 Meta-Learner: A regularized Logistic Regression classifier receives out-of-fold probability predictions from the level-0 models and computes optimal ensemble blend weights. This reduces variance and overcomes individual tree biases, yielding the highest overall ROC-AUC (**0.8048**).

Confusion Matrix & Threshold Analysis (Test Partition - 13,728 samples):

- **True Negatives (TN):** 5,400 | **False Positives (FP):** 1,537
- **False Negatives (FN):** 2,142 | **True Positives (TP):** 4,649
- **Optimal Clinical Threshold:** At $t = 0.40$, Sensitivity (Recall) rises to **76.56%** with 73.12% accuracy, ideal for medical screening scenarios where missing a diseased patient carries high penalty.

4. System Architecture & Full-Stack Integration

The project follows a decoupled, production-oriented client-server architecture:

Backend ML Service (FastAPI)	• Asynchronous ASGI server running on Uvicorn. • Pydantic schema validation for patient inputs. • Dynamic pipeline loader with joblib deserialization. • Multi-tiered fallback mechanism (Ensemble $\rightarrow$ Baseline $\rightarrow$ Clinical formula). • Standardized endpoints: /health, /predict, /docs.
Frontend Dashboard (React 18 + Vite)	• Component-driven UI with responsive navigation. • Real-time client-side BMI and blood pressure gauge calculation. • 1-click interactive demo profile presets for instant evaluation. • Transparent metric breakdowns (MAP, Pulse Pressure, Arterial Stiffness). • Resilient multi-endpoint fetch engine with proxy fallback.
How Backend & Frontend Connect	• The React frontend sends JSON payloads via HTTP POST to <code>http://127.0.0.1:8000/predict</code>. • FastAPI receives the request, validates the schema, applies FeatureEngineer, computes model probabilities, and returns a structured JSON response in <50ms. • Vite development proxy forwards /predict and /health seamlessly without CORS barriers.

5. Key Viva & Evaluation Q&A; (Defense Preparation)

Anticipated questions and technical answers for course evaluation:

Q1: Why did you use a Stacking Ensemble instead of a single algorithm?

A1: Single models like XGBoost or LightGBM have individual inductive biases (e.g. tree depth limits or split heuristics). Stacking uses out-of-fold cross-validation predictions from multiple distinct algorithms as meta-features into a regularized logistic meta-learner, minimizing generalization variance and boosting ROC-AUC to 0.8048.

Q2: How did you ensure there was no data leakage in feature engineering?

A2: All feature transformations (Pulse Pressure, MAP, log transformations, binning) are encapsulated inside an scikit-learn Pipeline using a custom TransformerMixin (FeatureEngineer). Scalers and estimators are fitted strictly on training folds and transformed on validation/test folds without sharing global statistics.

Q3: Why is ROC-AUC more clinically relevant than Accuracy?

A3: In cardiovascular disease screening, False Negatives (predicting a diseased patient as healthy) have severe consequences. ROC-AUC evaluates true positive rate vs false positive rate across all decision thresholds, allowing doctors to tune the sensitivity threshold (e.g. $t=0.40$) to catch 76.5% of cases.

Q4: How does the Logistic Regression from scratch work?

A4: It implements batch gradient descent from first principles using NumPy: $(z = Xw + b)$, sigmoid activation $(\sigma(z) = \frac{1}{1 + e^{-z}})$, binary cross-entropy loss gradients $(\frac{\partial L}{\partial w} = \frac{1}{m} X^T (\hat{y} - y))$, and iterative parameter updates $(w := w - \alpha \frac{\partial L}{\partial w})$. It achieved 72.70% accuracy, verifying scikit-learn's result.

6. Project Execution Commands

Component	Directory	Command to Run
Backend ML Service	ML_Project-main/ml-service	<code>.\.venv\Scripts\uvicorn api.app:app --reload --port 8000</code>
Frontend UI	ML_Project-main/frontend	<code>npm run dev</code> (Open <code>http://localhost:5173</code>)
Automated Tests	ML_Project-main/ml-service	<code>.\.venv\Scripts\pytest tests/test_api.py -v</code>
Retrain Ensemble	ML_Project-main/ml-service	<code>.\.venv\Scripts\python.exe training/train_advanced_models.py</code>